

目 录

第 1 章 实验概述	1
1.1 实验目的	1
1.2 实验环境	1
第 2 章 实验原理	2
2.1 冒泡排序算法	2
2.2 MIPS 汇编语言实现	2
2.3 定向 (Forwarding) 技术	3
2.4 静态调度优化	3
第 3 章 实验内容和步骤	4
3.1 实验内容	4
3.2 冒泡排序程序实现	4
3.2.1 程序实现	4
3.2.2 程序部分说明	6
3.2.3 寄存器使用说明	7
3.2.4 数据部分说明	7
3.2.5 程序执行结果	7
3.3 优化程序代码	9
3.3.1 优化思路	10
3.3.2 优化程序实现	10
3.3.3 优化程序执行结果	11
3.4 代码性能分析比较结果	12
第 4 章 实验总结与心得体会	14

第 1 章 实验概述

1.1 实验目的

1. 掌握静态调度方法
2. 增强汇编语言编程能力
3. 学会使用模拟器中的定向功能进行优化

1.2 实验环境

表 1.1 实验环境

配置项	参数
操作系统	Windows 11 24H2
仿真工具	MIPSSim v1.0.0 64-bit

第 2 章 实验原理

2.1 冒泡排序算法

冒泡排序是一种简单的排序算法，它的工作原理如下：

1. 比较相邻的元素。如果第一个比第二个大，就交换他们两个。
2. 对每一对相邻元素作同样的工作，从开始第一对到结尾的最后一对。在这一点，最后的元素应该会是最大的数。
3. 针对所有的元素重复以上的步骤，除了最后一个。
4. 持续每次对越来越少的元素重复上面的步骤，直到没有任何一对数字需要比较。

C 语言的一种实现方法如下：

```
1 void bubble_sort(int *arr, int len) {  
2     for (int i = 0; i < len - 1; i++) {  
3         for (int j = 0; j < len - i - 1; j++) {  
4             if (arr[j] > arr[j + 1]) {  
5                 // 交换 arr[j] 和 arr[j + 1]  
6                 int temp = arr[j];  
7                 arr[j] = arr[j + 1];  
8                 arr[j + 1] = temp;  
9             }  
10        }  
11    }  
12 }
```

2.2 MIPS 汇编语言实现

MIPS 汇编语言是一种简化的 RISC 架构，用于进行低级编程。本实验中，需要编写汇编程序来实现冒泡排序，其实现的基本思路如下：

- **数据加载：**使用 LW 指令从内存中加载数组和数组长度到寄存器中。
- **双层循环控制：**

- 外层循环控制排序的轮数，内层循环控制每轮的比较和交换。
- 使用 BEQ、BNE 等分支指令实现循环控制。
- **元素比较和交换**：将相邻元素加载至寄存器中，使用 SLT、BGT 等指令进行元素比较和交换。

2.3 定向 (Forwarding) 技术

定向技术通过将流水线寄存器中的中间计算结果直接“前递”到需要它的指令的 EX 段输入端，避免不必要的停顿，优化数据的流动，从而提升 CPU 的性能。在没有定向的情况下，某些指令需要等待前一指令完成并将结果写回寄存器后才能继续执行，这会导致停顿。而启用定向后，相关指令可以直接使用前一指令的结果，无需等待写回寄存器，从而减少停顿周期数。

2.4 静态调度优化

静态调度是在编译期（或手动编写时）重新排列指令顺序，以避免数据相关或资源冲突的技术。核心思想是将不相关的指令插入到可能导致停顿的位置，使流水线保持连续工作。典型的优化策略包括：

1. **重排相邻指令**：将与源数据无关的其他指令插入到 load 或运算指令之后，利用原本需要停顿的空隙执行有效工作。
2. **延迟槽填充**：将无害指令填入分支指令后的延迟槽，减少分支带来的性能损失。
3. **循环展开**：通过复制循环体减少循环控制开销，同时增加可用指令数以掩盖延迟。

第3章 实验内容和步骤

3.1 实验内容

1. 自行编写一个实现冒泡排序的汇编程序，该程序要求可以实现对一维整数数组进行冒泡排序。冒泡排序算法的运作如下：

1. 比较相邻的元素。如果第一个比第二个大，就交换他们两个。
2. 对每一对相邻元素作同样的工作，从开始第一对到结尾的最后一对。在这一点，最后的元素应该会是最大的数。
3. 针对所有的元素重复以上的步骤，除了最后一个。
4. 持续每次对越来越少的元素重复上面的步骤，直到没有任何一对数字需要比较。
5. 要求数组长度不得小于 10。

2. 启动 MIPSsim。

3. 载入自己编写的程序，观察流水线输出结果。

4. 使用定向功能再次执行代码，与刚才执行结果进行比较，观察执行效率的不同。

5. 采用静态调度方法重排指令序列，减少相关，优化程序

6. 对优化后的程序使用定向功能执行，与刚才执行结果进行比较，观察执行效率的不同。

注意：

1. 不要使用浮点指令及浮点寄存器！！
- 2 整数减勿使用 SUB 指令，请使用 DSUB 指令代替！！

3.2 冒泡排序程序实现

3.2.1 程序实现

我编写的冒泡排序汇编实现如下，注释在代码中已经给出：

```

1  .text
2  main:
3  ADDIU $r1, $r0, arr          # 数组 arr 的首地址
4  ADDIU $r2, $r0, len          # 长度 len 的地址
5  LW $r2, 0($r2)               # 数组长度 len
6  BGEZAL $r0, bubble_sort_func # 调用 bubble_sort_func
7  NOP                          # 延迟槽
8  TEQ $r0, $r0                 # 程序结束（触发异常停止）
9
10 # =====
11 # 冒泡排序函数 bubble_sort_func
12 # 输入：
13 #   $r1 = 数组 arr 首地址
14 #   $r2 = 数组长度 len
15 # =====
16 bubble_sort_func:
17 ADDIU $r3, $r2, -1           # $r3 = len - 1, 外层循环次数
18 BLEZ $r3, exit              # 如果 len <= 1, 无需排序
19
20 SLL $r2, $r2, 2             # $r2 = len * 4, 计算数组占用字节数
21 ADDIU $r4, $r1, 4           # $r4 = arr + 4, 数组第二个元素地址
22 ADDU $r5, $r1, $r2          # $r5 = arr + len*4, 数组末尾地址
23
24 outer_loop:                  # 外层循环，控制排序趟数
25 ADDIU $r6, $r4, 0           # $r6 = 当前比较元素地址，从第二个元素开始
26
27 inner_loop:
28 LW $r7, -4($r6)              # $r7 = arr[j-1], 加载前一个元素
29 LW $r8, 0($r6)               # $r8 = arr[j], 加载当前元素
30 SLT $r9, $r8, $r7           # 若 arr[j] < arr[j-1], 则 $r9 = 1
31 BEQ $r9, $r0, end           # 若不需要交换 (a[i] >= a[i-1]), 跳过
32
33 swap:                        # 交换元素
34 SW $r8, -4($r6)              # arr[j-1] = arr[j]
35 SW $r7, 0($r6)               # arr[j] = arr[j-1]
36

```

```

37 end:
38 ADDIU $r6, $r6, 4          # 移动到下一个元素地址
39 BNE $r5, $r6, inner_loop  # 如果未到达数组末尾，继续内层循环
40 ADDIU $r3, $r3, -1         # 外层循环次数减1
41 ADDIU $r5, $r5, -4         # 缩小边界（最后一个元素已排好）
42 BNE $r3, $r0, outer_loop  # 判断是否继续外层循环
43
44 exit:
45 JR $r31                   # 返回主程序
46
47 .data
48 arr: .word 23, 5, 91, 12, 47, 3, 68, 29, 74, 16, 8, 55, 39, 2, 60
49 len: .word 15

```

3.2.2 程序部分说明

程序主要分为两部分：主程序 main 和冒泡排序的函数 bubble_sort_func。主程序负责初始化寄存器并调用函数，而函数部分则实现了冒泡排序的逻辑，包括数据加载、循环控制和元素交换。

1. **初始化**：主程序首先将数组 arr 和长度 len 的地址加载到寄存器 \$r1 和 \$r2 中，然后通过 BGEZAL 指令跳转到冒泡排序函数 bubble_sort_func。
2. **函数实现**：在 bubble_sort_func 中，首先计算外层循环的次数 ($\text{len} - 1$)，并进行边界检查。如果数组长度小于或等于 1，则直接跳转到函数末尾返回。
4. **外层循环**：在 outer_loop 标签处，外层循环控制排序的趟数，每趟将最大的元素冒泡到末尾。每轮都将末尾地址减去一个元素宽度 (4 Bytes)，来缩小比较范围。
5. **内层循环**：在 inner_loop 标签处，内层循环负责比较和交换相邻元素。每次向前加载两个元素进行比较，如果需要交换，则执行交换操作。通过比较当前元素地址与末尾地址来判断是否继续。
5. **循环体结束判断**：当外层循环次数用尽后，程序跳转到 exit 标签处，通过 JR \$r31 返回主程序。
6. **程序结束**：主程序在函数返回后执行 TEQ \$r0, \$r0 指令结束程序。

本次实验我同样通过 BGEZAL + JR \$r31 的方式实现了**函数调用和返回**，其中：

- BGEZAL \$r0, bubble_sort_func: 这是一个条件跳转指令，表示如果 \$r0 大于或等于零（在这里 \$r0 始终为零），则跳转到 bubble_sort_func 标签处执行函数，并将返回地址存储在链接寄存器 \$r31 中。
- JR \$r31: 这是一个无条件跳转指令，表示跳转到 \$r31 寄存器中存储的地址处执行，这里用于函数返回。

3.2.3 寄存器使用说明

关于上述程序使用到的寄存器以及对应功能说明如表 3.1 所示。

表 3.1 MIPS 冒泡排序寄存器使用说明表

寄存器说明		
寄存器	作用	类型
r1	数组首地址 arr	输入
r2	数组长度 n / 字节数	输入/计算
r3	外层循环计数 (n-1)	控制
r4	第二个元素地址	指针
r5	当前内层循环边界	边界控制
r6	内层循环指针	指针
r7	a[i-1] 临时值	数据
r8	a[i] 临时值	数据
r9	比较结果 (SLT)	条件判断

3.2.4 数据部分说明

数据部分定义了数组 arr 和数组长度 len，数组 arr 为一个长度 15 的乱序数组。

3.2.5 程序执行结果

3.2.5.1 未开启定向的执行结果

载入并执行程序后，执行结果统计如图 3.1 所示，部分时间周期图如图 3.2 所示，程序共执行了 1707 个周期，其中发生了 677 次 RAW 停顿。总停顿周期占周期总数的 52.96%。

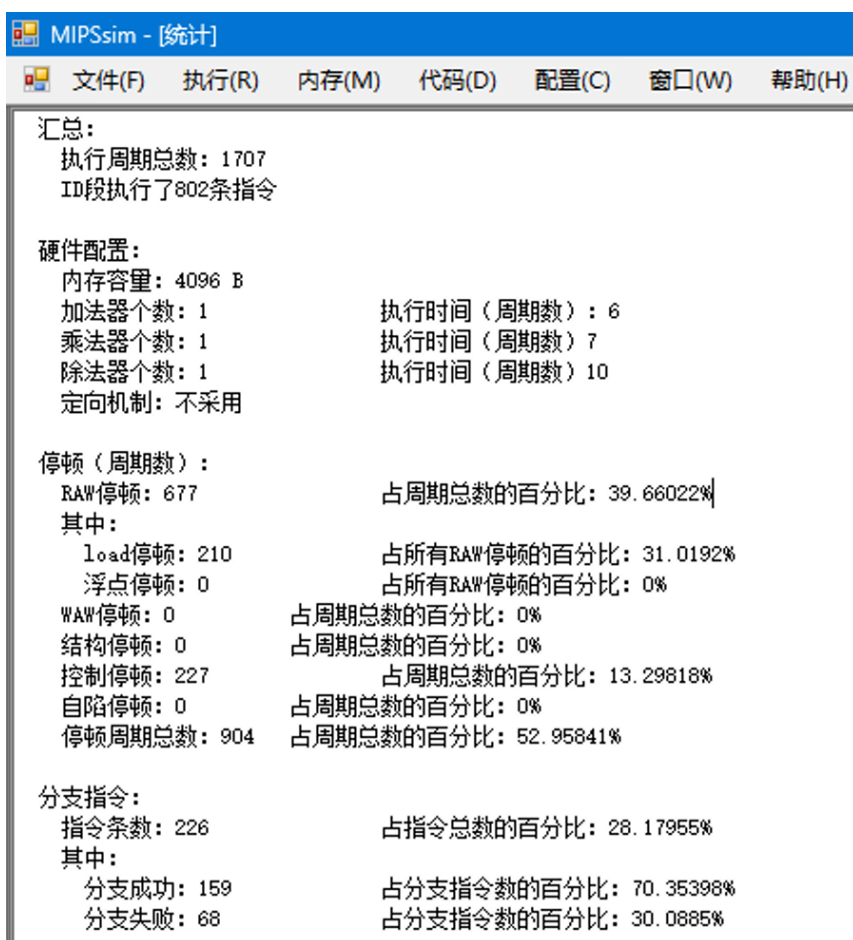


图 3.1 未开启定向的执行结果统计

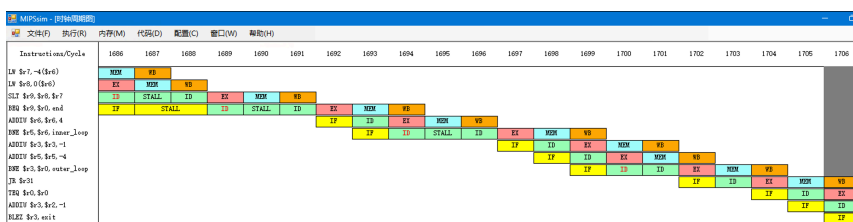


图 3.2 未开启定向的部分时间周期图

3.2.5.2 开启定向的执行结果

开启定向功能后再次执行程序，执行结果统计如图 3.3 所示，部分时间周期图如图 3.4 所示，程序共执行了 1242 个周期，其中发生了 316 次 RAW 停顿。总停顿周期占周

期总数的 43.72%，相比未开启定向时大幅减少，说明定向技术有效地优化了数据流动，提升了程序的执行效率。

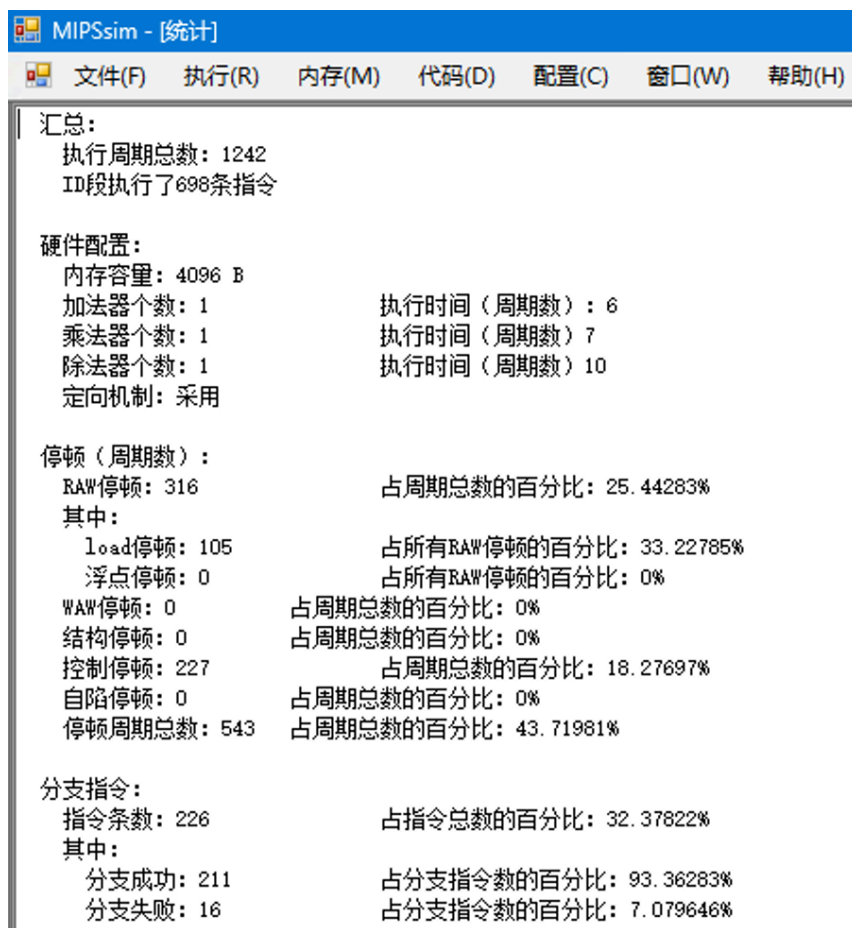


图 3.3 开启定向的执行结果统计

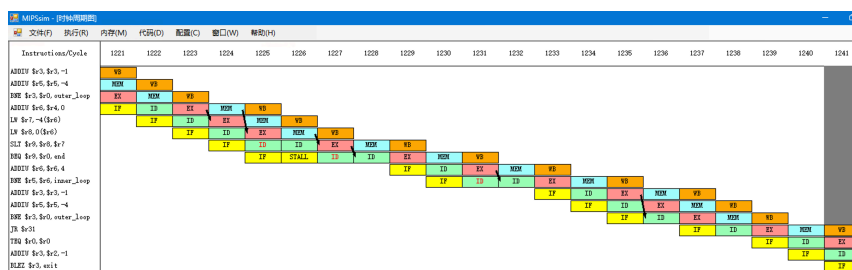


图 3.4 开启定向的部分时间周期图

3.3 优化程序代码

3.3.1 优化思路

针对上述程序中的数据相关问题，我采用了**静态调度**的方法对指令序列进行了重排，以减少 RAW 停顿的发生。原先的程序中：**SLT 指令依赖于前两条 LW 指令的结果**。在经典 5 级流水（IF/ID/EX/MEM/WB）中：

- LW 结果要到 MEM/WB 才可用
- SLT 在 EX 阶段就要用

根据静态调度原则，我们可以将 ADDIU 指令（更新内层循环指针）插入到 LW 和 SLT 之间，这样就可以利用这些指令的执行时间来掩盖 LW 的数据准备时间，从而避免 SLT 的停顿。

3.3.2 优化程序实现

```

1  .text
2  main:
3  ADDIU $r1, $r0, arr          # 数组 arr 的首地址
4  ADDIU $r2, $r0, len          # 长度 len 的地址
5  LW $r2, 0($r2)               # 数组长度 len
6  BGEZAL $r0, bubble_sort_func # 调用 bubble_sort_func
7  NOP                          # 延迟槽
8  TEQ $r0, $r0                 # 程序结束（触发异常停止）
9
10 # =====
11 # 冒泡排序函数 bubble_sort_func
12 # 输入：
13 #   $r1 = 数组 arr 首地址
14 #   $r2 = 数组长度 len
15 # =====
16 bubble_sort_func:
17 ADDIU $r3, $r2, -1           # $r3 = len - 1, 外层循环次数
18 BLEZ $r3, exit              # 如果 len <= 1, 无需排序
19
20 SLL $r2, $r2, 2              # $r2 = len * 4, 计算数组占用字节数
21 ADDIU $r4, $r1, 4            # $r4 = arr + 4, 数组第二个元素地址

```

```

22 ADDU $r5, $r1, $r2    # $r5 = arr + len*4, 数组末尾地址
23
24 outer_loop:           # 外层循环, 控制排序趟数
25 ADDIU $r6, $r4, 0     # $r6 = 当前比较元素地址, 从第二个元素开始
26
27 inner_loop:
28 LW $r7, -4($r6)       # $r7 = arr[j-1], 加载前一个元素
29 LW $r8, 0($r6)        # $r8 = arr[j], 加载当前元素
30 # 核心修改部分1: 将指令顺序调整, 提前执行指针更新
31 ADDIU $r6, $r6, 4     # 移动到下一个元素地址
32 # 核心修改部分1结束
33 SLT $r9, $r8, $r7     # 若 arr[j] < arr[j-1], 则 $r9 = 1
34 BEQ $r9, $r0, end     # 若不需要交换 (a[i] >= a[i-1]), 跳过
35
36 swap:                # 交换元素
37 # 核心修改部分2: 由于r6已经提前更新, 这里需要调整访问地址
38 SW $r8, -8($r6)       # arr[j-1] = arr[j]
39 SW $r7, -4($r6)       # arr[j] = arr[j-1]
40 # 核心修改部分2结束
41
42 end:
43 BNE $r5, $r6, inner_loop # 如果未到达数组末尾, 继续内层循环
44 ADDIU $r3, $r3, -1     # 外层循环次数减1
45 ADDIU $r5, $r5, -4     # 缩小边界 (最后一个元素已排好)
46 BNE $r3, $r0, outer_loop # 判断是否继续外层循环
47
48 exit:
49 JR $r31               # 返回主程序
50
51 .data
52 arr: .word 23, 5, 91, 12, 47, 3, 68, 29, 74, 16, 8, 55, 39, 2, 60
53 len: .word 15

```

3.3.3 优化程序执行结果

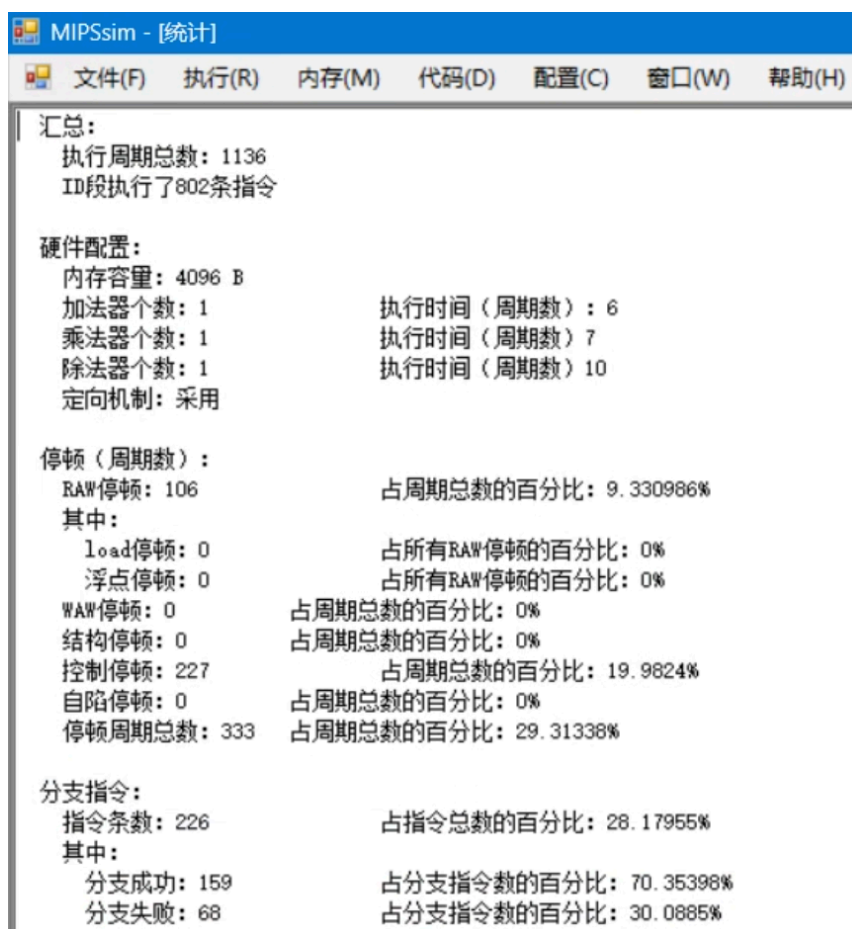


图 3.5 优化程序执行结果统计

开启定向功能后执行优化后的程序，执行结果统计如图 3.5 所示，部分时间周期图如图 3.6 所示，程序共执行了 1136 个周期，发生 106 次 RAW 停顿。总停顿周期数占周期总数的 29.31%。

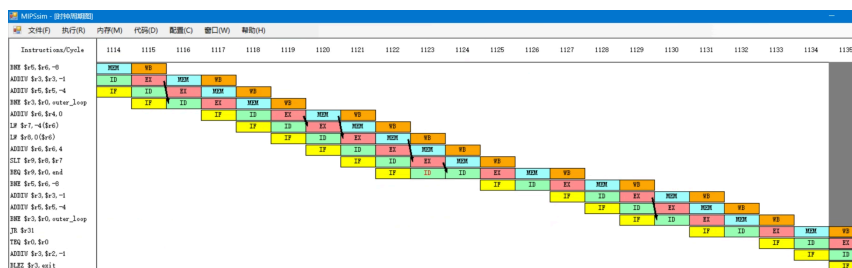


图 3.6 优化程序的部分时间周期图

3.4 代码性能分析比较结果

优化后的程序相比优化前的版本，执行周期数从 1707 个减少到 1136 个，停顿占比从 52.96% 降至 29.31%，效率提升至 $1707/1136 \approx 1.50$ 倍；

而相比仅开启定向的版本，执行周期数从 1242 个减少到 1136 个，停顿占比从 43.72% 降至 29.31%，效率提升至 $1242/1136 \approx 1.09$ 倍。

优化后的程序有了显著提升，主要得益于以下两点：

- 通过定向技术优化了数据流动；
- 通过静态调度优化指令顺序有效地减少了数据相关引起的停顿；

第4章 实验总结与心得体会

在本次实验中，我完成了冒泡排序的汇编代码编写，相比于向量点积程序，冒泡排序程序的指令数量和复杂度都有了显著增加，涉及了更多的寄存器使用和更复杂的双层循环控制结构。同时，我也通过定向技术和静态调度方法对程序进行了优化，显著提升了程序的执行效率。

通过本次实验，我对汇编语言的编程能力有了进一步的提升，尤其是在处理复杂控制结构 and 数据相关问题方面。同时，我也深刻理解了定向技术和静态调度方法在优化程序性能中的重要作用，以及它们在实际编程中的应用技巧。虽然冒泡排序的数据相关性较强，难以进行大幅优化，但本次实验仅通过移动一条无关指令就能有效减少停顿，提升性能，这让我对指令级并行和流水线优化有了更直观的认识。